

The advantages of algebraic design

When you design the algebra of a library first, representations for the data types of the algebra don't matter as much. As long as they support the required laws and functions, you don't even need to make your representations public.

There's an idea here that a type is given meaning based on its relationship to other types (which are specified by the set of functions and their laws), rather than its internal representation.⁷ This viewpoint is often associated with category theory, a branch of mathematics we've mentioned before. See the chapter notes for more on this connection if you're interested.

9.2 A possible algebra

We'll walk through the discovery of a set of combinators for the parsing tasks mentioned earlier. If you worked through this design task yourself, you likely took a different path and may have ended up with a different set of combinators, which is fine.

First, let's consider the parser that recognizes zero or more repetitions of the character 'a' and returns the number of characters it has seen. We can start by adding a primitive combinator for it; let's call it `many`:

```
def many[A] (p: Parser[A]): Parser[List[A]]
```

This isn't exactly what we're after—we need a `Parser[Int]` that counts the number of elements. We could change the `many` combinator to return a `Parser[Int]`, but that feels too specific—undoubtedly there will be occasions where we care about more than just the list length. Better to introduce another combinator that should be familiar by now, `map`:

```
def map[A,B] (a: Parser[A]) (f: A => B): Parser[B]
```

We can now define our parser like this:

```
map(many(char('a')))(_.size)
```

Let's add `map` and `many` as methods in `ParserOps`, so we can write the same thing with nicer syntax:

```
val numA: Parser[Int] = char('a').many.map(_.size)
```

We expect that, for instance, `run(numA) ("aaa")` gives `Right(3)`, and `run(numA) ("b")` gives `Right(0)`.

We have a strong expectation for the behavior of `map`—it should merely transform the result value if the `Parser` was successful. No additional input characters should be examined by `map`, and a failing parser can't become a successful one via `map` or vice

⁷ This sort of viewpoint might also be associated with object-oriented design, although OO hasn't traditionally placed much emphasis on algebraic laws. Furthermore, a big reason for encapsulation in OO is that objects often have some mutable state, and making this public would allow client code to violate invariants. That concern isn't relevant in FP.

versa. In general, we expect `map` to be *structure preserving* much like we required for `Par` and `Gen`. Let's formalize this by stipulating the now-familiar law:

```
map(p) (a => a) == p
```

How should we document this law? We could put it in a documentation comment, but in the preceding chapter we developed a way to make our laws *executable*. Let's use that library here.

Listing 9.2 Combining Parser with map

```
import fpinscala.testing._

trait Parsers[ParseError, Parser[+_]] {
  ...
  object Laws {
    def equal[A](p1: Parser[A], p2: Parser[A])(in: Gen[String]): Prop =
      forAll(in) (s => run(p1)(s) == run(p2)(s))

    def mapLaw[A](p: Parser[A])(in: Gen[String]): Prop =
      equal(p, p.map(a => a))(in)
  }
}
```

This will come in handy later when we test that our implementation of `Parsers` behaves as we expect. When we discover more laws later on, you're encouraged to write them out as actual properties inside the `Laws` object.⁸

Incidentally, now that we have `map`, we can actually implement `char` in terms of `string`:

```
def char(c: Char): Parser[Char] =
  string(c.toString) map (_.charAt(0))
```

And similarly another combinator, `succeed`, can be defined in terms of `string` and `map`:

```
def succeed[A](a: A): Parser[A] =
  string("") map (_ => a)
```

This parser always succeeds with the value `a`, regardless of the input string (since `string("")` will always succeed, even if the input is empty). Does this combinator seem familiar to you? We can specify its behavior with a law:

```
run(succeed(a))(s) == Right(a)
```

⁸ Again, see the chapter code for more examples. In the interest of keeping this chapter shorter, we won't give Prop implementations of all the laws, but that doesn't mean you shouldn't write them yourself!